Deep Reinforcement Learning

Advanced Algorithms (Part II)

Prof. Dr. Sebastian Peitz

Chair of Safe Autonomous Systems, TU Dortmund

Summer term 2026

Content

Content

- The evolution of modern RL algorithms
- Part (II): Improving Q-learning
  - Deterministic policy gradient
  - Deep deterministic policy gradient (DDPG)
  - TD3
  - Soft Actor-Critic

Where are we?

Chapter	Topic	Content
	Basics & tabular methods	
1-5	Bandits, MDPs, Dynamic Programming, Monte Carlo, TD Learning	RL basics in finite dimensions
	Deep-learning-based methods	
6	Brief introduction to deep learning	The basics for what comes next
7	Value function approximation	Value estimation with function approximation
8	Deep Q-learning	Q-learning with neural networks
9	Policy gradients	Direct optimization of the policy
10	Actor-critic algorithms	Improved policy gradients via value functions
11	Advanced algorithms (Part I): From policy gradient to PPO	The PG route to modern RL algorithms
12	Advanced algorithms (Part II): From Q-learning to Soft Actor-Critic	The AC route to modern RL algorithms
	Model-Based Control	
	Advanced Topics	

Lecture contents

The evolution of modern RL algorithms

The evolution of modern RL algorithms

(I) Policy gradient methods

- Policy is explicitly parameterized and optimized directly:

$$L(\phi) = \mathbb{E}_{\tau \sim p_\phi(\tau)} [r(\tau)].$$

- Gradient descent: $\phi \leftarrow \phi + \alpha \nabla_\phi L(\phi)$.
- Methods are typically **on-policy**.
- **Algorithms**: REINFORCE $\rightarrow$ Actor-Critic $\rightarrow$ Natural Policy Gradient $\rightarrow$ Trust-region policy optimization (TRPO) $\rightarrow$ Proximal policy optimization (PPO).

Shortcomings

- High variance gradients.
- Unstable policy updates.
- Catastrophic performance collapse.

Improvement strategy

- How to safely update policies.

(II) Value-based methods

- Approximation of the Q-function:

$$Q^*(s, a) = r + \max_{a' \in \mathcal{A}} Q^*(s', a').$$

- Extract implicit policy: $\pi(s) = \arg\max_{a \in \mathcal{A}} Q^*(s, a)$.
- Typically **off-policy**.
- **Algorithms**: Q-learning $\rightarrow$ DQN $\rightarrow$ Deep deterministic policy gradient (DDPG) $\rightarrow$ TD3 $\rightarrow$ Soft actor-critic (SAC).

Shortcomings

- Instability from bootstrapping.
- Overestimation bias.
- Maximization over actions / continuous action spaces.

Improvement strategy

- Stabilizing Q-learning with function approximation.

- Solving the continuous argmax problem.

Part (II): Improving Q-learning

Recall: Policy gradient and actor-critic

Policy gradient theorem – formulation via reward trajectories (sampling version in blue)

$$\nabla_\phi L(\phi) = \mathbb{E}_{\tau \sim p_\phi(\tau)} \left[\sum_{t=0}^{T-1} \nabla_\phi \log \pi_\phi(a_t|s_t) \left(\sum_{t'=t}^{T-1} r_{t'} \right) \right] \approx \frac{1}{N} \sum_{i=1}^{N} \left(\sum_{t'=t}^{T-1} \nabla_\phi \log \pi_\phi(a_{i,t}|s_{i,t}) \left(\sum_{t'=t}^{T-1} r_{i,t'} \right) \right).$$

Policy gradient theorem – formulation using the Q-function and the Actor-Critic architecture

$$\nabla_\phi L(\phi) = \mathbb{E}_{\tau \sim p_\phi(\tau)} \left[\sum_{t=0}^{T-1} \nabla_\phi \log \pi_\phi(a_t|s_t) A_\theta(s_t, a_t) \right] \approx \frac{1}{N} \sum_{i=1}^{N} \left(\sum_{t=0}^{T-1} \nabla_\phi \log \pi_\phi(a_{i,t}|s_{i,t}) A_\theta(s_{i,t}, a_{i,t}) \right).$$

Main drawbacks

1. **High-variance** gradient.
2. **Inefficient for continuous actions**, since sampling in high-dimensional settings becomes inefficient.
3. **On-policy** $\Rightarrow$ sample-inefficient (importance sampling makes problem 1. even worse!)

Approach: Find a more sample-efficient and off-policy capable version $\Rightarrow$ deterministic policy!

Off-policy policy gradients (1)

- (Degris, White, and Sutton 2012) presented an alternative form for the off-policy policy gradient (for finite $\mathcal{S}$ and $\mathcal{A}$) using an approximation. The starting point is the definition of the value function:

$$V^{\pi_\phi} = \sum_{s \in \mathcal{A}} \rho_\beta(s) \sum_{a \in \mathcal{A}} \pi_\phi(a|s) Q^{\pi_\phi}(s,a),$$

  where β is some behavior policy and $\rho_\beta(s) = \lim_{t \to \infty} p(s|s_0, \beta)$ is the limiting distribution of states when following β.

- Then, taking the gradient of V^{π_ϕ} with respect to ϕ, we get (via the product rule of differentiation):

$$\nabla_\phi V^{\pi_\phi} = \nabla_\phi \left[\sum_{s \in \mathcal{A}} \rho_\beta(s) \sum_{a \in \mathcal{A}} \pi_\phi(a|s) Q^{\pi_\phi}(s,a) \right] = \sum_{s \in \mathcal{A}} \rho_\beta(s) \sum_{a \in \mathcal{A}} [\nabla_\phi \pi_\phi(a|s) Q^{\pi_\phi}(s,a) + \pi_\phi(a|s) \nabla_\phi Q^{\pi_\phi}(s,a)].$$

- Ignoring the second part (i.e., ~~$\pi_\phi(a|s) \nabla_\phi Q^{\pi_\phi}(s,a)$~~ ; justification details in (Degris, White, and Sutton 2012)), we get

$$\nabla_\phi V^{\pi_\phi} \approx \sum_{s \in \mathcal{A}} \rho_\beta(s) \sum_{a \in \mathcal{A}} \nabla_\phi \pi_\phi(a|s) Q^{\pi_\phi}(s,a) = \mathbb{E}_{s \sim \rho_\beta} \left[\sum_{a \in \mathcal{A}} \nabla_\phi \pi_\phi(a|s) Q^{\pi_\phi}(s,a) \right].$$

💡 In contrast to the on-policy case (see the Actor-Critic lecture), here it is not possible to formulate a recursive formula which would allow us to find a closed-form statement for $\nabla_\phi Q^{\pi_\phi}(s,a)$ via $\nabla_\phi V^{\pi_\phi}(s)$. This only works in the on-polcy setting!

Off-policy policy gradients (2)

$$\nabla_\phi V^{\pi_\phi} \approx \sum_{s \in \mathcal{A}} \rho_\beta(s) \sum_{a \in \mathcal{A}} \nabla_\phi \pi_\phi(a|s) Q^{\pi_\phi}(s,a) = \mathbb{E}_{s \sim \rho_\beta} \left[\sum_{a \in \mathcal{A}} \nabla_\phi \pi_\phi(a|s) Q^{\pi_\phi}(s,a) \right].$$

- To make this off-policy w.r.t. the actions as well, we introduce *importance sampling* for β:

$$\nabla_\phi V^{\pi_\phi} \approx \sum_{s \in \mathcal{A}} \rho_\beta(s) \sum_{a \in \mathcal{A}} \beta(a|s) \underbrace{\frac{\pi_\phi(a|s)}{\beta(a|s)}}_{=\kappa_\phi(s,a)} \frac{\nabla_\phi \pi_\phi(a|s)}{\pi_\phi(a|s)} Q^{\pi_\phi}(s,a) = \sum_{s \in \mathcal{A}} \rho_\beta(s) \sum_{a \in \mathcal{A}} \beta(a|s) \kappa_\phi(s,a) \nabla_\phi \log \pi_\phi(a|s) Q^{\pi_\phi}(s,a)$$

$$= \mathbb{E}_{s \sim \rho_\beta, a \sim \beta(\cdot|s)} \left[\kappa_\phi(s,a) \nabla_\phi \log \pi_\phi(a|s) Q^{\pi_\phi}(s,a) \right].$$

- The starting point in (Silver et al. 2014) was very similar, but using continuous state and action spaces:

$$\nabla_\phi V^{\pi_\phi} \approx \int_{\mathcal{S}} \rho_\beta(s) \int_{\mathcal{A}} \kappa_\phi(s,a) \nabla_\phi \log \pi_\phi(a|s) Q^{\pi_\phi}(s,a) \, \mathrm{d}a \, \mathrm{d}s = \mathbb{E}_{s \sim \rho_\beta, a \sim \beta(\cdot|s)} \left[\kappa_\phi(s,a) \nabla_\phi \log \pi_\phi(a|s) Q^{\pi_\phi}(s,a) \right].$$

- We now have derived a simplified formula for the off-policy policy gradient.
- However, we still suffer from the large variance issue.

Deterministic policy gradient

Deterministic off-policy policy gradients

- Drop the randomness from the policy μ such that it becomes a **deterministic function**: $a = \mu_\phi(s)$.
- Reformulate V via Q: no need for expecations (i.e., sum / integrate over $\mathcal{A}$): $V^{\mu_\phi}(s) = Q^{\mu_\phi}(s, \mu_\phi(s))$.
- Deterministic: we are incapable of exploration! The theorem is only practically useful if we sample from off-policy data:

$$L(\phi) = \mathbb{E}_{s\sim\rho_\beta}\left[Q^{\mu_\phi}(s, \mu_\phi(s))\right] = \int_{\mathcal{S}} \rho_\beta(s) Q^{\mu_\phi}(s, \mu_\phi(s))\, \mathrm{d}s.$$

Deterministic policy gradient theorem (Silver et al. 2014)

Exact version: on-policy ($\rho_\beta = \rho_{\mu_\phi}$)

$$\begin{aligned}\nabla_\phi L(\phi) &= \nabla_\phi \left[\int_{\mathcal{S}} \rho_\beta(s) Q^{\mu_\phi}(s, \mu_\phi(s))\, \mathrm{d}s\right] = \int_{\mathcal{S}} \rho_\beta(s) \nabla_a Q^{\mu_\phi}(s, \mu_\phi(s))\big|_{a=\mu_\phi(s)} \nabla_\phi \mu_\phi(s)\, \mathrm{d}s \\ &= \mathbb{E}_{s\sim\rho_\beta}\left[\nabla_a Q^{\mu_\phi}(s, \mu_\phi(s))\big|_{a=\mu_\phi(s)} \nabla_\phi \mu_\phi(s)\right].\end{aligned} \tag{1}$$

Approximate version: off-policy ($\rho_\beta \neq \rho_{\mu_\phi}$, and we use the same simplification as earlier, i.e., ~~$\mu_\phi(a|s) \nabla_a Q^{\mu_\phi}(s, a)$~~)

$$\nabla_\phi L(\phi) \approx \mathbb{E}_{s\sim\rho_\beta}\left[\nabla_a Q^{\mu_\phi}(s, \mu_\phi(s))\big|_{a=\mu_\phi(s)} \nabla_\phi \mu_\phi(s)\right]. \tag{2}$$

💡 (1) is the limit case of the stochastic policy gradient theorem (i.e., $\mathbf{var}\left[\pi\right] \to 0$) (Silver et al. 2014, Theorem 2).

On-policy deterministic actor-critic

The simplest algorithm we can derive from this: SARSA-type on-policy Actor-Critic:

Algorithm: On-policy deterministic actor-critic

1. Sample $\{s_i, a_i, r_i, s_i', a_i'\}_{i=1}^N$ using $a = \mu_\phi(s)$.
2. TD error:

$$\delta_i = r_i + \gamma Q_\theta(s_i', a_i') - Q_\theta(s_i, a_i).$$

3. Semi-gradient Q-function update:

$$\theta \leftarrow \theta + \alpha_\theta \frac{1}{N} \sum_{i=1}^N \delta_i \nabla_\theta Q_\theta(s_i, a_i).$$

4. Update policy μ_ϕ by sampling (1):

$$\phi \leftarrow \phi + \alpha \frac{1}{N} \sum_{i=1}^N \nabla_a Q_\theta(s_i, a_i) \nabla_\phi \mu_\phi(s_i).$$

Summary of the concept

To update a deterministic policy off-policy, your algorithm does this for a batch of states from the replay buffer:

1. Pass state s into the Actor: $a = \mu_\phi(s)$.
2. Pass s and a into the Critic network $Q_\theta(s, a)$.
3. Compute how the Critic's output changes with respect to that action ($\nabla_a Q$).
4. Compute how the Actor's weights change to produce that action change ($\nabla_\phi \mu_\phi$).
5. Multiply them together to update the Actor.

Deep deterministic policy gradient (DDPG)

Deep deterministic policy gradient (DDPG)

Problems with vanilla DPG

The original algorithm assumed:

- tabular / linear approximators ("Compatible Function Approximation" in (Silver et al. 2014)),
- stable Q estimation.

When neural networks are used, several problems appear:

- Bootstrapping instability
- Correlated samples from trajectories
- Targets change too quickly
- Poor exploration due to deterministic policy

Deep deterministic policy gradient (DDPG) (Lillicrap et al. 2016)

addresses these by importing techniques from Deep Q-Networks (DQN).

Core components:

- Actor network $\mu_\phi(s)$
- Critic network $Q_\theta(s, a)$
- Target actor $\mu_{\phi'}$
- Target critic $Q_{\theta'}$
- Replay buffer

The DDPG algorithm

1. **Interact**: Sample $\{s_t, a_t, r_t, s_{t+1}\}$ using $a_t = \mu_\phi(s_t)$ (💡 *plus noise for exploration!*) and store in the replay buffer $\mathcal{D}$.
2. **Sample**: Draw random mini-batch of N transitions: $\mathcal{B} \subset \mathcal{D}$.
3. **Update critic**: Calculate the target y_i using the target networks (💡 *Optimal action $\mu_{\phi'}(s) \Rightarrow Q$-learning!*):

$$y_i = r_i + \gamma Q_{\theta'}(s_{i+1}, \mu_{\phi'}(s_{i+1})).$$

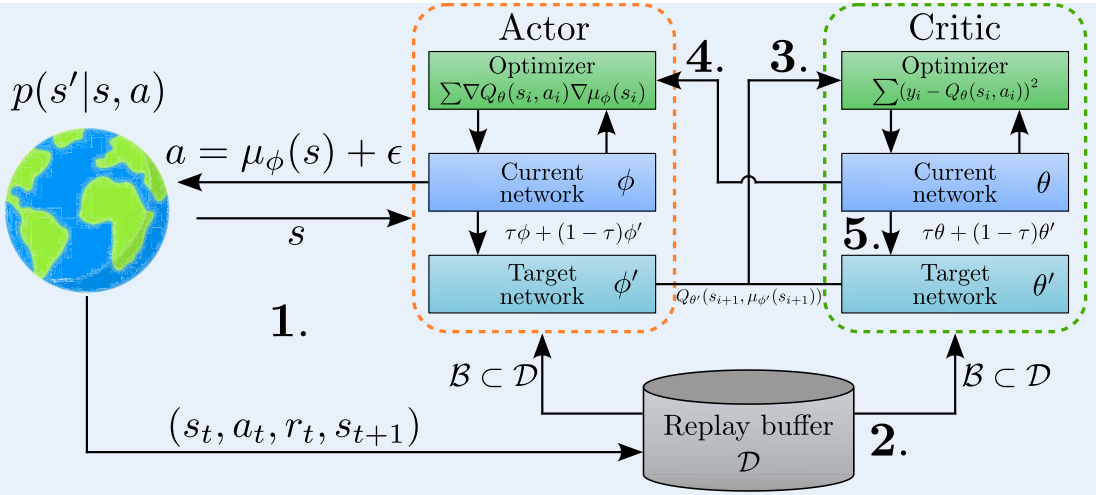
The *current critic* is updated by minimizing the Bellman error:

$$L(\theta) = \frac{1}{N} \sum_i \left(y_i - Q_\theta(s_i, a_i)\right)^2, \qquad \theta \leftarrow \theta + \alpha_\theta \frac{2}{N} \sum_{i=1}^{N} \left(y_i - Q_\theta(s_i, a_i)\right) \nabla_\theta Q_\theta(s_i, a_i).$$

4. **Update actor**: The *current actor* is updated using the sampled deterministic policy gradient (Eq. (1)):

$$\phi \leftarrow \phi + \alpha \frac{1}{N} \sum_{i=1}^{N} \nabla_a Q_\theta(s_i, a)\Big|_{a=\mu_\phi(s_i)} \nabla_\phi \mu_\phi(s_i).$$

5. **Soft updates**: Incremental target updates (ϕ' / θ').



The four main changes over DPG

1. **Integration of deep neural networks (CNNs)**
2. **Introduction of the replay buffer**

- Larger datasets to train from.
- Allows for i.i.d. sampling / breaks the temporal correlation of data.

3. **Explicit action noise for exploration**

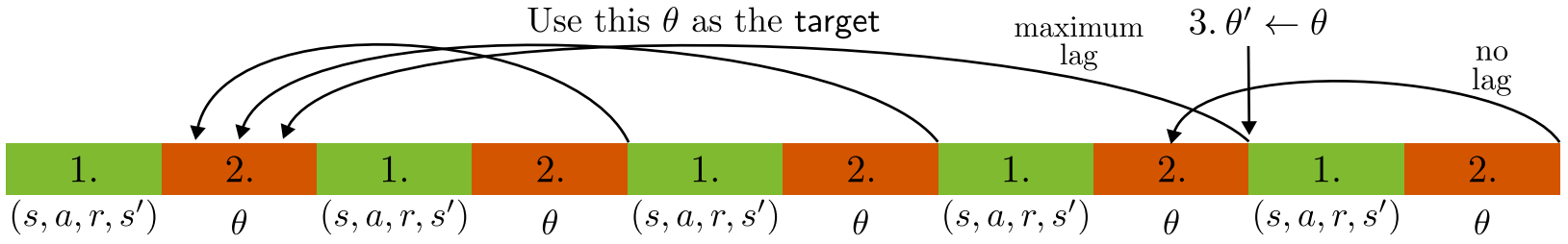
- Deterministic actor in DPG: it cannot explore on its own.
- DDPG adds an explicit random process $\mathcal{N}$ directly to the action selection during environment interaction.
- The original paper utilized Ornstein-Uhlenbeck noise, which creates temporally correlated, mean-reverting patterns that mimic inertial drift.

4. **Transition to "soft updates" for target networks**

- In DQN, target network weights are periodically copied exactly from the online network every few thousand steps.
- For continuous actor-critic configurations, hard updates changed the value landscape too abruptly.
- Instead: soft update, where target networks track the online networks smoothly at every single training step using an interpolation factor $\tau \ll 1$ (e.g., $\tau = 0.001$):

$$\theta' \leftarrow \tau\theta + (1-\tau)\theta', \qquad \phi' \leftarrow \tau\phi + (1-\tau)\phi'.$$

  $\Rightarrow$ targets change slowly, providing an unmoving baseline that stabilizes the deep network's gradients.
- We have seen this before under *Polyak averaging*.


Use this θ as the target
maximum lag
$3.\ \theta' \leftarrow \theta$
no lag
1.
2.
1.
2.
1.
2.
1.
2.
1.
2.
(s, a, r, s')
θ
(s, a, r, s')
θ
(s, a, r, s')
θ
(s, a, r, s')
θ
(s, a, r, s')
θ

Twin Delayed Deep Deterministic Policy Gradient (TD3)

Twin Delayed Deep Deterministic Policy Gradient (TD3)

DDPG works on many tasks, but **is highly sensitive** to hyperparameters as well as other randomness (e.g., the sampling).
$\Rightarrow$ In TD3 (Fujimoto, Hoof, and Meger 2018), three main issues were identified and addressed:

1. Maximization bias $\Rightarrow$ clipped double Q-learning

- Because the target uses a greedy step over actions $a = \mu_\phi(s)$, errors accumulate $\Rightarrow$ massive overestimation bias.
- Two independent critics (Q_{θ_1} / Q_{θ_2} and targets $Q_{\theta'_1}$ / $Q_{\theta'_2}$) and uses the minimum of their predictions to compute the target:

$$y = r + \gamma \min_{i=1,2} Q_{\theta'_i}(s', \mu_{\phi'}(s'))$$

2. Inaccurate critic $\Rightarrow$ delayed policy updates

- If the critic is highly inaccurate, updating the actor based on its gradients is counterproductive.
- Delaying the policy updates ensures the critic has reached a reliable value before the actor uses it.

$\Rightarrow$ Update actor (ϕ) and targets (ϕ', θ'_1, θ'_2) less frequently than critics (θ_1, θ'_2), e.g., every $N_a = 2$ steps.

3. Exploitation of artifacts in Q $\Rightarrow$ target action smoothing

- Deterministic policies are prone to exploiting sharp peaks or artifacts in the Q-function.
- Adding noise smooths out the value landscape, ensuring that similar actions yield similar values.

$\Rightarrow$ TD3 adds a small amount of clipped noise (clipping constant c) to the target action before feeding it into the target critic:

$$\tilde{a} = \mu_{\phi'}(s) + \epsilon, \quad \epsilon \sim \text{clip}(\mathcal{N}(0, \tilde{\sigma}^2), -c, c).$$

The TD3 algorithm

1. **Interact**: Sample $\{s_t, a_t, r_t, s_{t+1}\}$ using $a_t = \mu_\phi(s_t) + \epsilon$ ($\epsilon \sim \mathcal{N}\left(0, \sigma^2\right)$) and store in the replay buffer $\mathcal{D}$.
2. **Sample**: Draw random mini-batch of N transitions: $\mathcal{B} \subset \mathcal{D}$.
3. **Update critic**: Calculate targets y_i by minimizing over two target networks (💡 the **Twin**):

$$y_i = r_i + \gamma \min_{j \in \{1,2\}} Q_{\theta'_j}(s_{i+1}, \tilde{a}_{i+1}), \qquad \text{with noisy action } \tilde{a}_{i+1} = \mu_{\phi'}(s_{i+1}) + \epsilon, \quad \epsilon \sim \textbf{clip}(\mathcal{N}(0, \tilde{\sigma}^2), -c, c).$$

The two critics are updated by minimizing the Bellman errors:

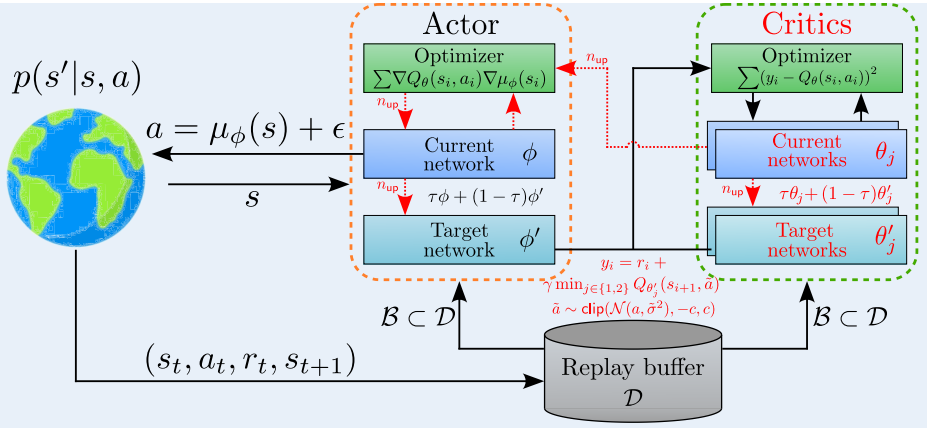
$$L(\theta_j) = \frac{1}{N} \sum_i \left(y_i - Q_{\theta_j}(s_i, a_i)\right)^2, \qquad \theta_j \leftarrow \theta_j + \alpha_\theta \frac{2}{N} \sum_{i=1}^{N} \left(y_i - Q_{\theta_j}(s_i, a_i)\right) \nabla_\theta Q_{\theta_j}(s_i, a_i).$$

Perform the following steps only every N_a^{th} step: (💡 the **Delayed**):

4. **Update actor**: The *online actor* is updated using the sampled deterministic policy gradient (Eq. (1)):

$$\phi \leftarrow \phi + \alpha \frac{1}{N} \sum_{i=1}^{N} \nabla_a Q_{\theta_1}(s_i, a)\Big|_{a=\mu_\phi(s_i)} \nabla_\phi \mu_\phi(s_i).$$

5. **Soft updates**: Incremental target updates (ϕ' / θ_1' / θ_2').



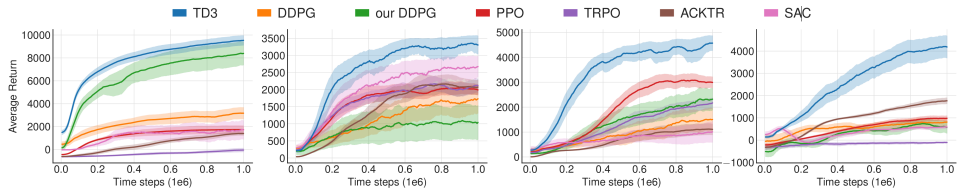
Soft actor-critic (SAC)

Soft actor-critic (SAC)

(Haarnoja et al. 2018)

Comparison

Example: Some MuJoCo environments



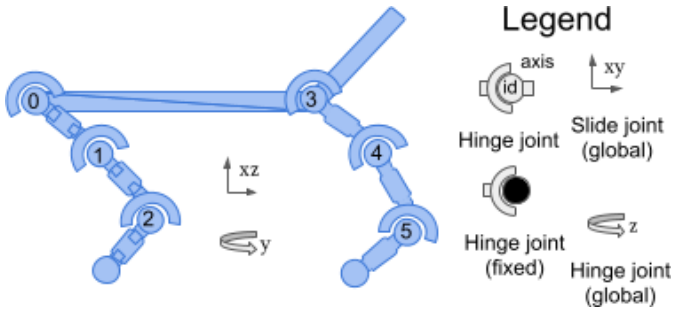
(a) HalfCheetah-v1 (b) Hopper-v1 (c) Walker2d-v1 (d) Ant-v1

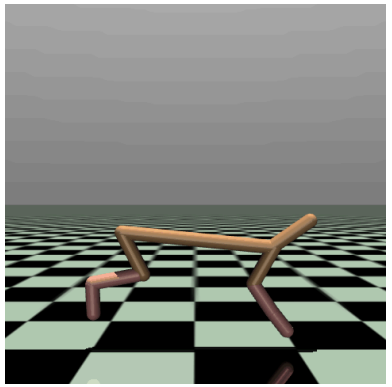
Source: (Fujimoto, Hoof, and Meger 2018, Figure 5).

Example: Half-Cheetah

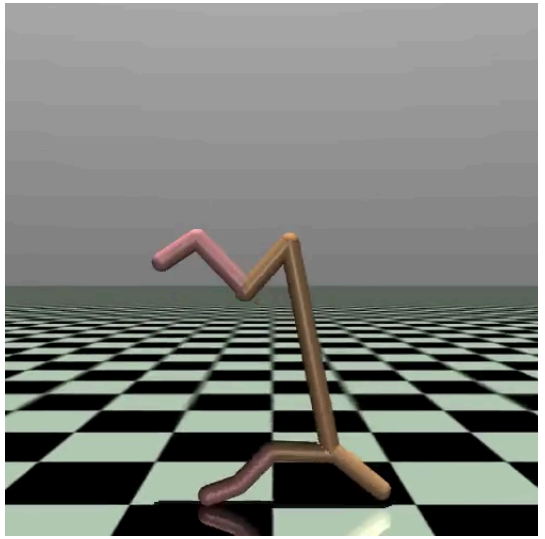
As an example, we study the Half Cheetah example from the MuJoCo library.

- $\mathcal{S}$: 17 states.
- $\mathcal{A}$: 6 actions.
- Reward: Forward-Cost - Control-Cost.

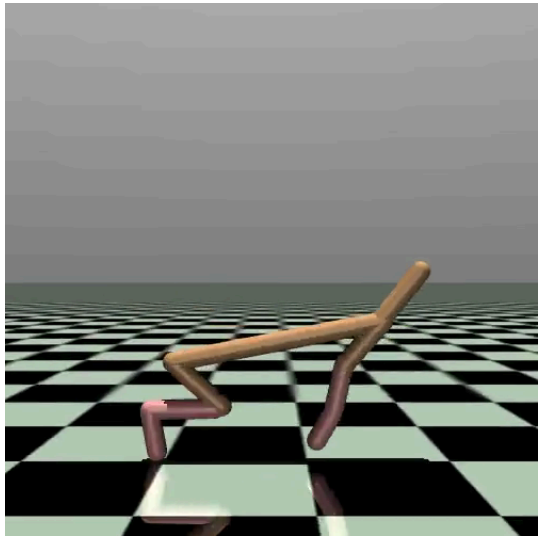




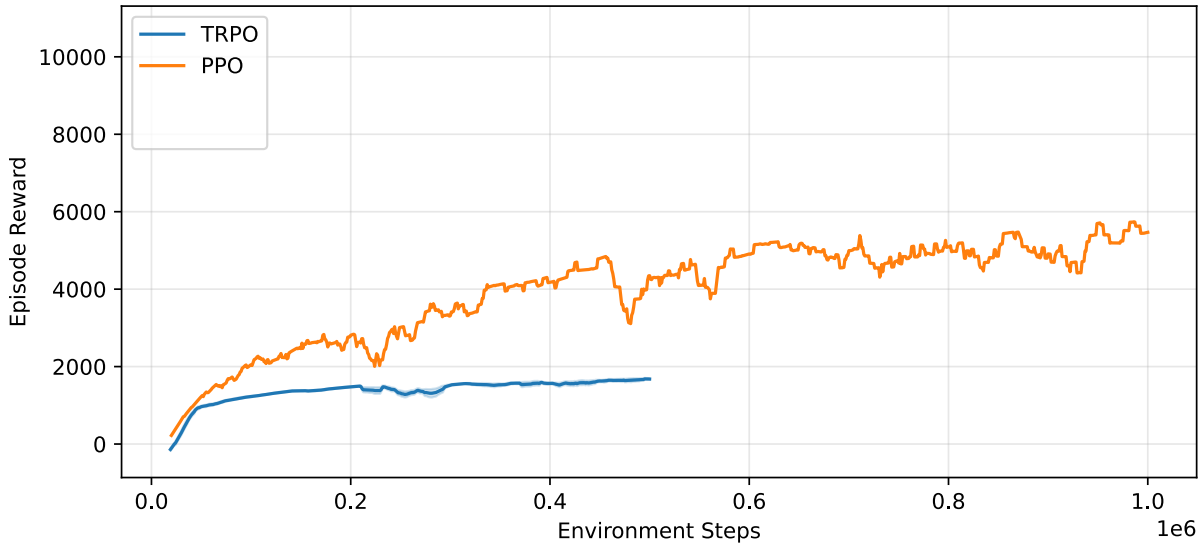
Results: TRPO vs. PPO (from the RL Baselines3 Zoo)



TRPO



PPO


TRPO
PPO
Episode Reward
10000
8000
6000
4000
2000
0
0.0
0.2
0.4
0.6
0.8
1.0
1e6
Environment Steps

References

Degris, Thomas, Martha White, and Richard S. Sutton. 2012. “Off-Policy Actor-Critic.” In *International Conference on Machine Learning*.

Fujimoto, Scott, Herke van Hoof, and David Meger. 2018. “Addressing Function Approximation Error in Actor-Critic Methods.” In *Proceedings of the 35th International Conference on Machine Learning*, edited by Jennifer Dy and Andreas Krause, 80:1587–96. Proceedings of Machine Learning Research. PMLR.

Haarnoja, Tuomas, Aurick Zhou, Pieter Abbeel, and Sergey Levine. 2018. “Soft Actor-Critic: Off-Policy Maximum Entropy Deep Reinforcement Learning with a Stochastic Actor.” In *Proceedings of the 35th International Conference on Machine Learning*, edited by Jennifer Dy and Andreas Krause, 80:1861–70. Proceedings of Machine Learning Research. PMLR.

Lillicrap, Timothy P., Jonathan J. Hunt, Alexander Pritzel, Nicolas Heess, Tom Erez, Yuval Tassa, David Silver, and Daan Wierstra. 2016. “Continuous Control with Deep Reinforcement Learning.” In *International Conference on Learning Representations (ICLR)*.

Silver, David, Guy Lever, Nicolas Heess, Thomas Degris, Daan Wierstra, and Martin Riedmiller. 2014.